MockKernel — Frontend Team Roadmap & Work Division

> **Context:** The Backend (`Server/` & `Worker/`) is being completed and maintained by your backend peer.
> This roadmap divides **100% of the Frontend (`Client/`)** equally between **Developer 1 (You)** and **Developer 2 (Your Teammate)** so both of you can work in parallel on independent branches without merge conflicts or blockers.

---

1. High-Level Work Split at a Glance

Area	Developer 1 (You) — Arena & Live Contests	Developer 2 (Teammate) — AI Voice, Dashboard & Admin
Domain	Competitive Coding, Monaco Editor, Execution Pipeline, Contests & Realtime Leaderboards	AI Mock Interviewer, Voice Integration, Student Dashboard, Real Auth & Admin/SuperAdmin Control Room
Shared Tech / Core	Socket.io client setup, Submissions polling fallback, Split-pane layout, Editor drafts	Base API client (`client.ts`), Reusable UI Primitives (`Modal` , `Toast` , `DataTable` , `Tabs`), Role Guards
Pages Owned	1. Problem Catalog (`/practice`) 2. Problem Workspace & Editor (`/problems/:id`) 3. Contests Hub (`/contests`) 4. Live Contest Room (`/contests/:id`) 5. Live Leaderboard (`/contests/:id/leaderboard`)	1. AI Interview Setup & Room (`/interview` , `/interview/:id`) 2. Interview Scorecard & History (`/interview/:id/summary` , `/interviews`) 3. Student Dashboard (`/dashboard`) 4. Admin Overview & Problems CRUD (`/admin` , `/admin/problems`) 5. Contests Control Room, Plagiarism & Users (`/admin/contests` , `/admin/users`)
Edge Cases Owned	**E2:** 10s TLE Timeout explanation UI **E4:** Redis 429 submit spam cooldown ring	**E1:** Mic denied / unsupported manual textarea fallback **E3:** Groq 429 "AI is busy, retrying..." banner
Estimated Effort	50%	50%

---

2. Shared Foundation (Quick Prep Before Branching)

Both developers should ensure these packages are installed in `Client/` before starting:

```
cd Client
npm install @monaco-editor/react socket.io-client
```

And fix the existing TypeScript error in `Client/src/components/ui/VerdictBadge.tsx` by adding `RUNNING` (tone `run`) and `INTERNAL_ERROR` (tone `bad`) to the `config` record.

---

3. Developer 1 (You) — Scope & Deliverables

Focus: Coding Workspace, Monaco Editor, Async Execution & Realtime Contests

3.1 Components & Modules to Build

1. `VerdictBadge.tsx` **Fix:**
   - Add missing `RUNNING` and `INTERNAL_ERROR` entries so `npm run build` passes.
2. `ResizablePanels.tsx` **:**
   - Draggable horizontal and vertical split containers for problem statement vs. Monaco Editor vs. output console.
3. `Timer.tsx` **:**
   - Monospaced countdown timer for active contests with warning state when `< 5 minutes` remain.
4. **Socket.io Client Hook (`src/hooks/useSocket.ts`):**
   - Connect to backend socket with `{ auth: { token } }` .

- Track connection status (`connected`, `reconnecting`, `polling`).
- Listen for `verdict:update` and `leaderboard:update`.
- Provide `contest:join` and `contest:leave` emitters.
- **Automatic Polling Fallback:** If socket drops or disconnects, automatically poll `GET /api/submissions/:id/status` every 1.5s until a terminal verdict is reached.

3.2 Pages & Features

1. **Problem Catalog (`/practice` or `/problems`):**
   - Wire up `GET /api/problems` with real search, difficulty filter chips (`Easy`, `Medium`, `Hard`), and pagination.
   - Glass cards with problem title, topic tag, acceptance rate, and difficulty badge.
2. **Problem Workspace & Coding Arena (`/problems/:id`):**
   - **Left Panel:** Problem statement, constraints, input/output format, sample test cases with copy buttons, and problem submission history tab.
   - **Right Panel (Monaco Editor):**
     - Monaco dark theme matching `#161B22` / transparent glass.
     - Language selector: Python, C++, Java, JavaScript.
     - Auto-load problem `starterCode` per language.
     - Local draft persistence in `localStorage` + warning modal before leaving with unsaved code.
   - **Bottom Console / Verdict Panel:**
     - Tabs: `Test Cases` (inputs/expected outputs) and `Output` (live execution results).
     - Optimistic `PENDING` status shown within 100ms of clicking Run/Submit.
     - Live state updates via Socket.io (`RUNNING` → `ACCEPTED` / `WRONG_ANSWER` / `TLE` / `COMPILE_ERROR`).
     - Stderr / stdout display and formatted compile error messages.
     - **Edge Case E2:** Explanatory alert on TLE: "*Your code exceeded the 10-second limit — check for infinite loops or inefficient logic.*"
     - **Edge Case E4:** On HTTP 429, show toast message and disable Run/Submit buttons with a visual cooldown countdown ring.
     - **Plagiarism Status:** If verdict is `PLAGIARISM_FLAGGED`, show neutral student badge "*Under review*" (never show similarity scores to students).
3. **Contests List (`/contests`):**
   - List `live` and `ended` contests (`GET /api/contests`).
   - Display start/end time, problem count, participant count, and `Join Contest` button (`POST /api/contests/:id/join`).
4. **Live Contest Arena (`/contests/:id`):**
   - Live contest HUD: big countdown timer, problem navigation tabs (`Problem A`, `B`, `C`), participant solved status dots.
   - Integrated Monaco editor and submission flow scoped to `contestId`.
   - `Leave Contest` confirmation modal (`POST /api/contests/:id/leave`) only permitted if no solutions have been submitted yet.
5. **Live Contest Leaderboard (`/contests/:id/leaderboard`):**
   - Realtime table showing Rank, Participant Name, Total Score, and Solved count.
   - Live row position re-ordering animations upon `leaderboard:update` socket events.
   - Highlight current user's row + bottom sticky card if the user is off-screen.

---

4. Developer 2 (Your Teammate) — Scope & Deliverables

Focus: AI Voice Interview, Student Dashboard, Auth API & Admin/SuperAdmin Control Room

4.1 Components & Modules to Build

1. **Base API Client (`src/api/client.ts`):**
   - Standard fetch/axios wrapper injecting `Authorization: Bearer <accessToken>`.
   - Global error handling that extracts `{ error: { code, message } }`.
   - Auto-logout on `401 Unauthorized`.
   - Rate-limit response header inspection (`X-RateLimit-Remaining`, `Retry-After`).
2. **Shared UI Primitives (`src/components/ui/`):**
   - `Modal.tsx`: Glassmorphic dialog with backdrop blur, keyboard `Escape` closing, and focus trap.
   - `Toast.tsx` / `ToastContext`: Top-right notifications for errors, info, and success alerts.

- `DataTable.tsx`: Reusable table with glass rows, column sorting, pagination controls, and empty/loading skeletons.
- `Skeleton.tsx`: Shimmer loading placeholder matching Neon Glass palette.
- `EmptyState.tsx`: Icon + heading + description + primary action button.
- `Tabs.tsx` & `Drawer.tsx`: Reusable tab switchers and slide-over side drawers for forms.

3. **Role Guards & Route Security (`src/components/auth/RouteGuards.tsx`):**
   - Add `RequireRole({ roles: ['admin', 'superadmin'] })` route guard.
   - Redirect unauthorized users to `/` with a warning toast.
   - Update `Navbar.tsx` to conditionally display `Dashboard`, `Interview`, `Practice`, `Contests`, and `Admin Panel` links based on `user.role`.

4.2 Pages & Features

1. **Real Authentication Wire-up (`src/api/auth.ts`):**
   - Replace the `localStorage` mock in `src/api/auth.ts` with real endpoints:
     - `POST /api/auth/register`
     - `POST /api/auth/login`
     - `POST /api/auth/google`
     - `GET /api/auth/me`
     - `POST /api/auth/logout`
   - Store access token and user role in Zustand `authStore`, and hydrate on app load via `GET /api/auth/me`.
2. **AI Voice Interview Room & Flow (`/interview` & `/interview/:id`):**
   - **Setup Screen (`/interview`):**
     - Select Topics (`Operating Systems`, `DBMS`, `Data Structures`).
     - Select Difficulty (`Easy`, `Medium`, `Hard`).
     - Select Question Count (`5`, `8`, `10`).
     - `Start Interview` button calls `POST /api/interviews` and navigates to `/interview/:sessionId`.
   - **Active Interview Room (`/interview/:id`):**
     - Central animated **Voice Orb** (pulsing gradient via Framer Motion that scales with audio input).
     - Progress indicator (e.g. `Question 3 of 8`).
     - Display current AI question in readable typography.
     - **Voice Input:** Use browser Web Speech API (`SpeechRecognition` / `webkitSpeechRecognition`) with live transcript display.
     - `Submit Answer` button calls `POST /api/interviews/:id/answer`.
     - Displays per-answer AI score (`0-10`) and constructive feedback before adapting to the next question.
     - **Edge Case E1 (Hardware / Permission Fallback):** If mic permission is denied or Web Speech API is unsupported, immediately display an inline glass banner and slide open a clean manual textarea for typed answers without breaking the session.
     - **Edge Case E3 (AI Rate Limit / Busy):** If Groq returns 429 or is retrying, display a friendly `"AI interviewer is thinking, retrying in a moment..."` banner; never discard the student's answer.
     - Action buttons: `Complete Early` (`POST /api/interviews/:id/complete`) and `Abandon` (`PATCH /api/interviews/:id/abandon`).
3. **Interview Scorecard & History (`/interview/:id/summary` & `/interviews`):**
   - **Summary Screen:** Shows overall score out of 10, performance feedback badge, and accordion review of all questions, candidate answers, scores, and AI suggestions.
   - **History Screen (`/interviews`):** Past interview sessions list (`GET /api/interviews`) with date, topics, difficulty, and score.
4. **Student Dashboard (`/dashboard`):**
   - Welcome banner with student name and avatar.
   - Quick KPI stat cards: Problems Solved, Contests Participated, Interviews Completed, Acceptance Rate.
   - Quick Action cards: "*Start Mock Interview*", "*Solve Daily Problem*", "*Browse Live Contests*".
   - Recent activity list showing recent submissions and recent interview results.
5. **Admin & SuperAdmin Suite (`/admin/*`):**
   - **Overview Dashboard (`/admin`):** Platform stats from `GET /api/admin/stats` (Total Problems, Published Problems, Live Contests, Submissions, Registered Users).
   - **Problems Manager (`/admin/problems`):**
     - Paginated `DataTable` of all problems with published/draft badges.
     - Create & Edit Drawer/Modal: Title, Statement (Markdown), Constraints, Difficulty dropdown, Languages multi-select, Starter code editor per language.
     - Interactive Test Cases Editor: Add/remove ordered test cases, sample flag toggle, input and expected output textareas.
     - Delete problem confirmation modal (handled safely via API transaction).

- **Contests Manager (`/admin/contests`):**
  - List all contests with status tags (`draft`, `live`, `ended`).
  - Create contest form: Title, description, start/end dates, problem selection with points weighting.
  - Contest Action buttons: `Start Contest` (`POST /start`), `End Contest` (`POST /end`), `Rebuild Leaderboard` (`POST /leaderboard/rebuild`).
- **Live Contest Monitor & Plagiarism Viewer (`/admin/contests/:id/live`):**
  - Live submissions feed and participants list.
  - Flagged Submissions tab (`GET /api/admin/contests/:id/flagged`): view similarity percentages and matched submission code diffs.
  - `Disqualify Participant` button with destructive confirmation modal (`PATCH /disqualify`).
- **SuperAdmin User Management (`/admin/users`):**
  - Restricted to `role === 'superadmin'`.
  - Filterable user table (search by name/email, filter by role).
  - `Create Admin` modal form (`POST /api/admin/users`) to provision new admin accounts.

---

5. Shared API & Realtime Reference Cheat Sheet

Common Request Header

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Backend Endpoints Overview

Module	Route	Method	Auth / Role	Description
Auth	`/api/auth/register`	`POST`	Public	Register student (`{ name, email, password }`)
	`/api/auth/login`	`POST`	Public	Login with email & password
	`/api/auth/google`	`POST`	Public	Login with Google ID token (`{ idToken }`)
	`/api/auth/me`	`GET`	Authenticated	Current profile (`{ id, name, email, role }`)
	`/api/auth/logout`	`POST`	Authenticated	Logout
Problems	`/api/problems`	`GET`	Public	List published problems (`page`, `limit`, `search`, `difficulty`)
	`/api/problems/:id`	`GET`	Public	Get problem statement & sample test cases
	`/api/admin/problems`	`GET` / `POST`	Admin	List all problems / Create problem + test cases
	`/api/admin/problems/:id`	`GET` / `PATCH` / `DELETE`	Admin	Get full problem / Update / Delete
Submissions	`/api/submissions/run`	`POST`	Authenticated	Run code against sample tests (`202 Accepted`)
	`/api/submissions/submit`	`POST`	Authenticated	Submit code against all tests (`202 Accepted`)

	`/api/submissions/:id/status`	`GET`	Authenticated	Poll submission status & test case outputs
	`/api/submissions`	`GET`	Authenticated	User's submission history
Contests	`/api/contests`	`GET`	Optional Auth	List live & ended contests
	`/api/contests/:id`	`GET`	Optional Auth	Contest details with problem list
	`/api/contests/:id/join`	`POST`	Authenticated	Join a live contest
	`/api/contests/:id/leave`	`POST`	Authenticated	Leave before submitting
	`/api/contests/:id/leaderboard`	`GET`	Optional Auth	Contest leaderboard rankings
	`/api/admin/contests`	`GET` / `POST`	Admin	Admin contest list & creation
	`/api/admin/contests/:id/status`	`PATCH`	Admin	Lifecycle transition (`draft` → `live` → `ended`)
	`/api/admin/contests/:id/leaderboard/rebuild`	`POST`	Admin	Rebuild Redis leaderboard from Mongo
	`/api/admin/contests/:id/flagged`	`GET`	Admin	View plagiarism flagged submissions
	`/api/admin/contests/:id/participants/:userId/disqualify`	`PATCH`	Admin	Disqualify participant
Interviews	`/api/interviews`	`POST`	Authenticated	Start session (`topics` , `difficulty` , `questionCount`)
	`/api/interviews/:id/answer`	`POST`	Authenticated	Submit answer, get evaluation & next question
	`/api/interviews/:id/complete`	`POST`	Authenticated	Complete active session & generate summary
	`/api/interviews/:id/abandon`	`PATCH`	Authenticated	Abandon session
	`/api/interviews/:id`	`GET`	Authenticated	Get session transcript & scores
	`/api/interviews`	`GET`	Authenticated	List user's past interviews
Admin	`/api/admin/stats`	`GET`	Admin	Platform counts
	`/api/admin/users`	`GET` / `POST`	SuperAdmin	List users / Create Admin user

Socket.io Events

- **Server to Client:**
  - `verdict:update` → Payload: `{ submissionId, verdict, runtime, memory, compileOutput, testCaseResults }`
  - `leaderboard:update` → Payload: `{ contestId, entries: [{ rank, userId, name, score }], pagination }`
- **Client to Server:**
  - `contest:join` → Emits contest ID string

- `contest:leave` → Emits contest ID string

---

6. Ready-to-Paste Master Prompt for Her (Teammate) & Her AI Assistant

> **How to use:** Copy everything inside the code block below and give it to your teammate. She can paste it directly into her AI assistant (Antigravity, Cursor, Claude, ChatGPT, etc.) at the start of her turn.

```
You are a senior frontend engineer and UI/UX expert working on the "MockKernel" project.
I am new to the codebase, so please guide me clearly, write production-ready TypeScript code, explain key decisions, and never make assumptions without checking.

========================================
1. PROJECT CONTEXT & ARCHITECTURE
========================================
MockKernel is an AI-powered mock interview and competitive coding platform.
- Backend: Express 5 + TypeScript + MongoDB + Redis + BullMQ + Judge0 CE + Groq API (Llama-3.3). The backend is already built by our peer team on port 3000 (http://localhost:3000).
- Frontend: React 19 + TypeScript + Vite + Tailwind CSS + Zustand + Lucide React + Framer Motion.
- Monorepo folder for frontend: /Client

Our team is split into two frontend engineers:
- Developer 1 owns: The Monaco Code Editor workspace, execution submission pipeline, live contests, and live leaderboards.
- Developer 2 (ME) owns: The UI component primitives, API client, real Auth integration, AI Voice Mock Interviewer, Student Dashboard, and the Admin / SuperAdmin Suite.

========================================
2. DESIGN SYSTEM: THEME C — "NEON GLASS"
========================================
We strictly follow "Theme C: Neon Glass" from Documentations/screens.md.pdf.
All styles must strictly use our Tailwind design tokens and CSS variables. NEVER use hardcoded hex values inside components.

Key Design Tokens (already defined in tailwind.config.ts):
- Background: base = #07070F (dark with subtle violet/cyan radial gradient blobs)
- Panels & Cards: Glass panels with `bg-white/5 backdrop-blur-xl border border-white/10`
- Text: ink = #F5F5FF (primary), muted = #9CA3C7 (secondary)
- Accent Gradients: brand = linear-gradient(135deg, #7C3AED, #06B6D4) (Violet to Cyan)
- Accent Colors: violet = #7C3AED, cyan = #06B6D4, pink = #F472B6
- Status / Verdict Colors:
  - ok (Accepted) = #34D399
  - bad (Wrong Answer / Compile Error / Error) = #FB7185
  - warn (TLE / Rate Limit) = #FBBF24
  - run (Pending / Running) = #60A5FA
  - flag (Plagiarism / Under review) = #C084FC
- Typography:
  - Headings: Space Grotesk Variable (font-display)
  - Body: Inter Variable (font-sans)
  - Code / Numbers: JetBrains Mono Variable (font-mono)
- Radii: card = 16px (rounded-card), controls = 12px (rounded-ctl), pills = rounded-full
- Shadows: glow = 0 0 24px rgba(124,58,237,.35), glow-cyan = 0 0 24px rgba(6,182,212,.30)
- Motion: Framer Motion spring easing (250-350ms), smooth pulse on active voice orb, respectful of prefers-reduced-motion.

========================================
3. MY EXACT ASSIGNED SCOPE & DELIVERABLES
========================================

--- TASK 1: Base API Client & Auth Integration ---
1. Create `src/api/client.ts`:
   - Use `fetch` or `axios` configured with `baseURL = import.meta.env.VITE_API_URL || 'http://localhost:3000'`.
```

```
   - Attach `Authorization: Bearer <token>` from Zustand `authStore`.
   - Handle global errors, format backend `{ error: { code, message } }`, and auto-logout on 401.
2. Wire real backend in `src/api/auth.ts`:
   - Replace the localStorage mock with real API calls:
     - POST /api/auth/register
     - POST /api/auth/login
     - POST /api/auth/google
     - GET /api/auth/me
     - POST /api/auth/logout
   - Update `authStore.ts` to manage token + user profile and hydrate on mount.
3. Role Guard:
   - Create `RequireRole` in `src/components/auth/RouteGuards.tsx` to guard `/admin/*` routes for roles 'admin' and
'superadmin'.
   - Update `Navbar.tsx` so students see links for Practice, Contests, Interview, and Dashboard; while admins also see an "Admin
Panel" link.

--- TASK 2: Reusable UI Component Primitives ---
Build in `src/components/ui/` with full keyboard accessibility (WCAG AA), glassmorphism, and responsive design:
- `Modal.tsx`: Accessible dialog with backdrop blur and Escape key dismissal.
- `Toast.tsx`: Toast provider and context for top-right notifications.
- `DataTable.tsx`: Translucent table with sortable columns, pagination controls, and empty/skeleton states.
- `Skeleton.tsx`: Shimmer placeholder for loading states.
- `EmptyState.tsx`: Clean empty state illustration/icon + headline + action.
- `Tabs.tsx`: Animated tab switcher.
- `Drawer.tsx`: Slide-over drawer for forms.

--- TASK 3: AI Voice Mock Interview Room & Summary ---
Routes: `/interview` (setup), `/interview/:id` (room), `/interview/:id/summary` (results), `/interviews` (history)
1. Setup Screen (`/interview`):
   - Choose Topics: 'Operating Systems', 'DBMS', 'Data Structures'.
   - Choose Difficulty: 'Easy', 'Medium', 'Hard'.
   - Choose Question Count: 5, 8, 10.
   - Start button calls POST /api/interviews -> returns sessionId and first question.
2. Active Room (`/interview/:id`):
   - Hero animated Voice Orb in the center (pulsing gradient with Framer Motion).
   - Question counter (e.g. "Question 2 of 5") and topic badge.
   - AI Question card with typewriter or smooth fade-in.
   - Speech Recognition: Web Speech API (SpeechRecognition / webkitSpeechRecognition) capturing spoken answers with live
transcript.
   - Answer submission: POST /api/interviews/:id/answer -> returns `{ evaluation: { score, feedback }, nextQuestion, isComplete
}`.
   - Show evaluation score (0-10) and feedback pill after each turn.
   - Mandatory Edge Case E1 (Mic Denied / Unsupported): If microphone access is denied or unsupported, show an inline glass
banner and slide open a clean manual textarea so the user can type answers without breaking the interview.
   - Mandatory Edge Case E3 (AI Rate Limit / Busy): If Groq returns 429 or is retrying, display a friendly "AI interviewer is
thinking, retrying in a moment..." indicator without losing the user's transcript.
   - Buttons for "Complete Interview" (POST /complete) and "Abandon" (PATCH /abandon).
3. Summary Screen (`/interview/:id/summary`):
   - Overall score gauge (e.g. 8.5 / 10), performance verdict, strengths & weaknesses summary.
   - Full question-by-question accordion showing Question, Student Answer, Score, and Feedback.
4. Past History (`/interviews`):
   - List past sessions with date, topics, difficulty, and score.

--- TASK 4: Student Dashboard ---
Route: `/dashboard`
- Welcome banner with student name and avatar.
- Metric KPI tiles: Problems Solved, Contests Participated, Interviews Taken.
- Quick action buttons ("Practice Code", "Take AI Interview", "View Contests").
- Recent activity feed showing recent submissions and completed interviews.

--- TASK 5: Admin & SuperAdmin Control Room ---
Routes: `/admin`, `/admin/problems`, `/admin/contests`, `/admin/contests/:id/live`, `/admin/users`
```

1. Overview (`/admin`):
   - Display system stats from GET /api/admin/stats (Total Problems, Published Problems, Live Contests, Total Submissions, Total Users).
2. Problems Manager (`/admin/problems`):
   - DataTable listing problems with title, difficulty, languages, and published status.
   - Problem Create/Edit Drawer: Form for title, statement (markdown), constraints, difficulty, supported languages, starter code per language.
   - Test Cases sub-editor: Add/remove ordered test cases (input, expected output, isSample flag, explanation).
   - Delete confirmation modal (calls DELETE /api/admin/problems/:id).
3. Contests Manager (`/admin/contests`):
   - Contest list with status tags: Draft, Live, Ended.
   - Contest Create Drawer: Title, description, start time, end time, problem picker with points per problem.
   - Lifecycle controls: "Start Contest", "End Contest", "Rebuild Leaderboard" buttons.
4. Live Contest Monitor & Plagiarism Viewer (`/admin/contests/:id/live`):
   - Live participants list and submissions feed.
   - Flagged Submissions tab (GET /api/admin/contests/:id/flagged): displays similarity percentage and matched submission comparison.
   - Disqualify participant modal (calls PATCH /api/admin/contests/:id/participants/:userId/disqualify).
5. SuperAdmin User Manager (`/admin/users`):
   - Only accessible if user.role === 'superadmin'.
   - Filterable table of all users with role badges.
   - "Create Admin" modal form calling POST /api/admin/users.

==
4. CODING RULES & INSTRUCTIONS
==
1. Work step-by-step. Do not generate all files at once; propose each step, write clean code, and verify type safety.
2. All components must be in TypeScript (.tsx) with strict typing. Never use `any`.
3. Use Tailwind CSS utility classes using our Theme C tokens (e.g. `bg-base`, `text-ink`, `text-muted`, `bg-brand`, `bg-white/5`, `backdrop-blur-xl`, `border-white/10`, `rounded-card`, `rounded-ctl`).
4. Ensure full keyboard navigation, aria-labels on icon buttons, and responsive layouts.
5. If anything is unclear, ask me before proceeding.

Let's begin with TASK 1 (Base API client, real auth integration, and RouteGuards). Tell me what files we will create or edit first!